

Занятие 3.4 Лабораторная работа.

Формирование детального слоя данных (DDS) с использованием Apache Spark: очистка, преобразование, объединение и загрузка данных в целевое хранилище.

ЧАСТЬ 1. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

Цель работы.

Освоить технологии распределённой обработки больших данных с использованием фреймворка Apache Spark (PySpark) для формирования детального слоя данных (DDS) в рамках архитектуры Data Lakehouse: от чтения сырых данных из централизованного хранилища (Data Lake) до их очистки, преобразования, объединения и загрузки в целевое табличное хранилище на базе Apache Iceberg поверх распределённой файловой системы HDFS.

Проверяемые компетенции.

Код	Индикатор	Уровень	
BD-4.1	Осуществляет выбор технологий обработки больших данных, приемлемых для создания прикладной системы ИИ с заданными требованиями	Продвинутый	Организует централизованное хранилище данных (Data Lake), их распределённое хранение, параллельную обработку, а также обработку потоковых данных
PL-1.3	Разрабатывает и поддерживает системы обработки больших данных различной степени сложности	Средний	Способен применять основные функции фреймворка PySpark, может самостоятельно построить процесс обработки больших данных с использованием Airflow

Дескриптор освоения: Организует централизованное хранилище данных (Data Lake), их распределенное хранение, параллельную обработку, а также обработку потоковых данных.

Задание

1. Подключиться к кластеру Hadoop/Spark, настроить сессию Spark с интеграцией Apache Iceberg.
2. Загрузить сырые данные из распределённого хранилища (HDFS) в Spark DataFrame из нескольких источников.
3. Выполнить **очистку данных**: обработка пропущенных значений, приведение типов, удаление дубликатов, фильтрация аномалий.
4. Выполнить **преобразование данных**: фильтрация, агрегация, применение оконных функций, создание производных признаков.
5. Выполнить **объединение данных** из нескольких источников с использованием различных типов JOIN.
6. Провести **проверки качества данных (Data Quality)**: полнота, уникальность, согласованность.
7. Загрузить обработанные данные **в целевое хранилище** (таблица Apache Iceberg в HDFS). При создании таблицы явно указать format-version='2', обосновав этот выбор как обязательное условие для поддержки будущей инкрементальной (поточковой) записи данных (Streaming) в формате Iceberg.
8. Подготовить отчёт о проделанной работе.

Варианты заданий (по номеру студента в группе mod 4)

Исходные данные:

Студенты самостоятельно скачивают исходный "сырой" датасет US Used Cars Dataset с платформы Kaggle по ссылке:
https://www.kaggle.com/datasets/ananyamital/us-used-cars-dataset?select=used_cars_data.csv

Примечание: Студент обязан самостоятельно выполнить загрузку данных в HDFS, очистку от пропусков и дубликатов, преобразование типов данных и фильтрацию выбросов (включая обоснованное применение

ограничений, указанных выше), а также формирование новых признаков (например, объединение признаков сертификации в один бинарный признак `is_any_cert` и расчет возраста автомобиля `age`). Только после этого выполняется задание по варианту.

Вариант 0. Обогащение данных справочником дилеров

Синтетический справочник: Студент генерирует с помощью PySpark таблицу `synthetic_dealers` (поля: `dealer_id`, `dealer_name`, `region`, `dealer_rating` (1.0–5.0), `established_year`). Для связывания студент добавляет в основной датасет случайный `dealer_id` (от 1 до 500).

Логика объединения: LEFT JOIN основного датасета со справочником `synthetic_dealers` по `dealer_id`.

Ключевые операции преобразования:

1. Оконные функции: расчет среднего рейтинга дилера и средней цены автомобиля в разрезе `region` с использованием `Window.partitionBy()`.
2. Создание производных признаков: `price_vs_region_avg` (отклонение цены автомобиля от средней по региону), `dealer_experience_years` (стаж дилера).

Итоговая DDS-таблица: `dds_cars_with_dealers`.

Партиционирование: по году продажи (извлекается из признака `year`) и месяцу. Формат: Apache Iceberg.

Вариант 1. Обогащение данных справочником регионов и климата

Синтетический справочник: Студент генерирует таблицу `synthetic_regions` (поля: `region_code`, `region_name`, `climate_zone` (Cold, Temperate, Hot), `avg_price_index` (0.8–1.3)). Для связывания студент создает в основном датасете признак `region_code`

Логика объединения: INNER JOIN основного датасета со справочником `synthetic_regions` по `region_code`.

Ключевые операции преобразования:

1. Бакетизация (Bucketing): использование Bucketizer из `pySpark.ml.feature` для разделения числового признака `mileage` на 5 категорий (например, 0–50k, 50k–100k и т.д.).
2. Преобразование признаков: расчет скорректированной цены $\text{adjusted_price} = \text{price} * \text{avg_price_index}$.
3. Оконные функции: ранжирование автомобилей по пробегу внутри каждой группы `climate_zone` с помощью `row_number().over(Window.partitionBy("climate_zone").orderBy("mileage"))`.

Итоговая DDS-таблица: `dds_cars_by_region`.

Партиционирование: по `climate_zone` и `body_type`. Формат: Apache Iceberg.

Вариант 2. Формирование витрины страховых рисков

Синтетический справочник: Студент генерирует таблицу `synthetic_insurance` (поля: `risk_level` (Low, Medium, High), `avg_premium`, `coverage_type`).

Логика объединения: Студент самостоятельно создает в основном датасете признак `risk_level` на основе правил (например, `body_type == 'SUV' и horsepower > 250 → 'High', horsepower < 150 → 'Low', иначе → 'Medium'`). Далее выполняется LEFT JOIN со справочником по `risk_level`.

Ключевые операции преобразования:

1. Агрегация: группировка данных для подсчета количества автомобилей и средней цены (`avg(price)`) для каждой комбинации `risk_level` и `wheel_system`.
2. Оконные функции: расчет кумулятивной суммы (нарастающим итогом) базовой страховой премии (`sum(avg_premium).over(Window.partitionBy("risk_level").orderBy("age"))`).
3. Создание производного признака: `is_high_risk_vehicle` (бинарный флаг).

Итоговая DDS-таблица: dds_cars_insurance_risk.

Партиционирование: по risk_level и body_type. Формат: Apache Iceberg.

Вариант 3. Анализ комплектаций и опций (Options Analysis)

Синтетический справочник: Студент генерирует таблицу synthetic_options_catalog (поля: option_name, category (Safety, Comfort, Tech), estimated_value).

Логика объединения:

1. Использование функции explode на столбце major_options (массив строк) для перевода массива в отдельные строки.
2. INNER JOIN развернутого датасета со справочником synthetic_options_catalog по option_name.
3. Обратная агрегация (groupBy("vin").agg(...)) для сбора статистики по каждому автомобилю.

Ключевые операции преобразования:

1. Агрегация: подсчет общего количества опций (count), суммы оценочной стоимости опций (sum(estimated_value)) и сбор списка уникальных категорий в массив (collect_set(category)) для каждого vin.
2. Оконные функции: вычисление процента ранга (percent_rank) автомобиля по количеству опций внутри своего body_type.
3. Создание производного признака: total_estimated_value = price + sum_estimated_options.

Итоговая DDS-таблица: dds_cars_options_enriched.

Партиционирование: по body_type и is_any_cert. Формат: Apache Iceberg.

Общие требования для всех вариантов:

В отчете студент должен явно продемонстрировать этапы очистки "сырых" данных (обоснование удаления выбросов по указанным выше порогам, обработка пропусков, удаление дубликатов по vin), логику

генерации синтетического справочника, текст SQL/PySpark-запросов для объединения и оконных функций, а также скриншоты или вывод команд, подтверждающие успешную запись партиционированной таблицы в формате Apache Iceberg и проверку ее структуры.

Контрольные вопросы

1. Что такое детальный слой данных (DDS) в архитектуре хранилища данных? Какова его роль в Data Lake?
2. В чём отличие между архитектурами Data Lake и Data Warehouse? Какие преимущества даёт их комбинация (Lakehouse)?
3. Какие компоненты Apache Spark используются для распределённой обработки данных? Опишите архитектуру Spark-приложения (Driver, Executor, Cluster Manager).
4. Что такое Apache Iceberg? Какие преимущества он даёт по сравнению с традиционными форматами хранения (Parquet, ORC)?
5. Какие типы JOIN поддерживает PySpark? В чём особенности их распределённого выполнения?
6. Что такое партиционирование и bucketing в контексте распределённых хранилищ? Как они влияют на производительность запросов?
7. Какие показатели качества данных (Data Quality) вы проверяли в ходе выполнения работы? Как их автоматизировать?
8. Что такое оконные функции в PySpark? Приведите примеры их применения в ETL-процессах.
9. Какие инструменты оркестрации (например, Apache Airflow) могут использоваться для автоматизации ETL-процессов и как они интегрируются с PySpark?
10. Как преобразовать данный пакетный (batch) ETL-конвейер в потоковый (streaming) с использованием Spark Structured Streaming? Опишите ключевые отличия в API (readStream / writeStream) и

необходимые изменения в инфраструктуре (например, механизм checkpointing для обеспечения отказоустойчивости).

ЧАСТЬ 2. МЕТОДИЧЕСКИЕ УКАЗАНИЯ

1. Теоретические основы

1.1. Архитектура Data Lake и детальный слой данных (DDS)

Data Lake представляет собой централизованное хранилище, позволяющее хранить все структурированные и неструктурированные данные в любом масштабе. В классической многоуровневой архитектуре хранилища выделяют следующие слои:

- Raw Layer (Бронзовый слой) — сырые данные без изменений
- DDS / Clean Layer (Серебряный слой) — очищенные, стандартизированные, интегрированные данные
- Data Marts / Feature Store (Золотой слой) — агрегированные витрины данных для аналитики и ML

Детальный слой данных (DDS) представляет собой слой, на котором данные проходят первичную очистку, стандартизацию и интеграцию из различных источников, но сохраняют детальный (атомарный) уровень granularity.

В современных реалиях классическая архитектура Data Lake эволюционирует в архитектуру Lakehouse (Data Lake + Data Warehouse). Использование табличного формата Apache Iceberg позволяет сохранить масштабируемость и гибкость Data Lake для хранения сырых данных (Raw Layer), одновременно обеспечивая надежность, ACID-транзакции и структурированность, необходимые для формирования качественного детального слоя данных (DDS / Silver Layer).

1.2. Apache Spark для обработки больших данных

Apache Spark — фреймворк для распределённой обработки больших данных. Ключевые компоненты:

- Spark DataFrame — распределённая коллекция данных, организованная в именованные колонки
- Spark SQL — модуль для работы со структурированными данными через SQL-запросы
- Spark Core — базовый движок с поддержкой RDD и планировщиком задач
- PySpark — Python API для Spark

Архитектура Spark-приложения:

- Driver — координирует выполнение, хранит контекст приложения
- Executor — выполняет задачи на рабочих узлах
- Cluster Manager (YARN, Kubernetes, Standalone) — управляет ресурсами

1.3. Apache Iceberg

Apache Iceberg представляет собой открытый табличный формат для больших аналитических наборов данных. Ключевые возможности:

- ACID-транзакции для операций с данными
- Схемная эволюция (добавление/удаление/переименование колонок)
- Time Travel — доступ к историческим версиям данных
- Скрытое партиционирование — автоматическое управление партициями

- Интеграция с различными движками обработки (Spark, Trino, Flink)

1.4. Показатели качества данных (Data Quality)

Ключевые метрики качества данных:

- Полнота (Completeness) — доля заполненных значений
- Уникальность (Uniqueness) — отсутствие дубликатов
- Согласованность (Consistency) — соответствие бизнес-правилам
- Актуальность (Timeliness) — своевременность данных
- Корректность (Validity) — соответствие формату и диапазону

2. Подготовка окружения

2.1. Подключение к кластеру

```
# Подключение к узлу кластера  
ssh hadoop-node
```

```
# Переход в рабочую директорию  
cd ~/student_workspace/lab2
```

```
# Активация виртуального окружения  
source ~/pyspark-env/bin/activate
```

2.2. Настройка Spark-сессии

Для работы с Apache Iceberg необходимо настроить SparkConf с указанием необходимых пакетов:

```
from pyspark.sql import SparkSession  
from pyspark import SparkConf  
import os
```

```
def create_spark_configuration() -> SparkConf:
```

```
    """
```

```
    Создает и конфигурирует экземпляр SparkConf для приложения Spark  
    с интеграцией Apache Iceberg.
```

```
    Returns:
```

```
        SparkConf: Настроенный экземпляр SparkConf.
```

```
    """
```

```
    user_name = os.getenv("USER")
```

```
    conf = SparkConf()
```

```
    conf.setAppName("Lab2_DDS_Formation")
```

```
    conf.setMaster("yarn")
```

```
    conf.set("spark.submit.deployMode", "client")
```

```

conf.set("spark.executor.memory", "12g")
conf.set("spark.executor.cores", "8")
conf.set("spark.executor.instances", "2")
conf.set("spark.driver.memory", "4g")
conf.set("spark.driver.cores", "2")

# Подключение Apache Iceberg
conf.set("spark.jars.packages",
        "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.6.0")
conf.set("spark.sql.extensions",
        "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
conf.set("spark.sql.catalog.spark_catalog",
        "org.apache.iceberg.spark.SparkCatalog")
conf.set("spark.sql.catalog.spark_catalog.type", "hadoop")
conf.set("spark.sql.catalog.spark_catalog.warehouse",
        f"hdfs:///user/{user_name}/warehouse")
conf.set("spark.sql.catalog.spark_catalog.io-impl",
        "org.apache.iceberg.hadoop.HadoopFileIO")
return conf

```

```

conf = create_spark_configuration()
spark = SparkSession.builder.config(conf=conf).getOrCreate()

```

3. Ход работы

Этап 1. Загрузка сырых данных из нескольких источников

```

# Установка текущей базы данных
database_name = "student_database"
spark.catalog.setCurrentDatabase(database_name)

# Загрузка сырых данных из нескольких источников
raw_transactions = spark.table("raw_transactions")
raw_users = spark.table("raw_users")
raw_events = spark.table("raw_events")

# Анализ схемы и объёма данных

```

```
raw_transactions.printSchema()
print(f"Transactions count: {raw_transactions.count()}")
print(f"Users count: {raw_users.count()}")
print(f"Events count: {raw_events.count()}")
```

Задание для студента: Проанализировать схему данных каждого источника, определить типы признаков, выявить пропущенные значения и аномалии.

Этап 2. Очистка данных

```
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType, IntegerType, TimestampType
from pyspark.sql.window import Window
```

```
def clean_dataframe(df, columns_config):
```

```
    """
```

Выполняет очистку DataFrame: приведение типов, обработка пропусков, удаление дубликатов.

Args:

df: Исходный DataFrame.

columns_config: Словарь с конфигурацией колонок.

Returns:

DataFrame: Очищенный DataFrame.

```
    """
```

```
    # Приведение типов
```

```
    for col_name, col_type in columns_config.get("types", {}).items():
```

```
        df = df.withColumn(col_name, F.col(col_name).cast(col_type))
```

```
    # Обработка пропущенных значений
```

```
    fill_values = columns_config.get("fill_values", {})
```

```
    if fill_values:
```

```
        df = df.fillna(fill_values)
```

```

# Удаление дубликатов
subset_cols = columns_config.get("dedup_subset", None)
df = df.dropDuplicates(subset=subset_cols)

return df

# Конфигурация очистки для транзакций
transactions_config = {
    "types": {
        "amount": DoubleType(),
        "user_id": IntegerType(),
        "timestamp": TimestampType()
    },
    "fill_values": {
        "category": "unknown",
        "amount": 0.0
    },
    "dedup_subset": ["transaction_id", "timestamp"]
}

cleaned_transactions = clean_dataframe(raw_transactions, transactions_config)

```

Задание для студента: Реализовать стратегию очистки для каждого источника данных, обосновать выбор методов обработки пропущенных значений.

Этап 3. Преобразование данных и оконные функции

```

# Применение оконных функций для агрегации
window_spec = (Window
    .partitionBy("user_id")
    .orderBy(F.col("timestamp"))
    .rowsBetween(Window.unboundedPreceding, Window.currentRow))

# Расчёт накопительной суммы и количества транзакций

```

```
transformed_transactions = (cleaned_transactions
    .withColumn("cumulative_amount",
        F.sum("amount").over(window_spec))
    .withColumn("transaction_count",
        F.count("*").over(window_spec))
    .withColumn("days_since_first",
        F.datediff(F.col("timestamp"),
            F.min("timestamp").over(
                Window.partitionBy("user_id"))))
    )
```

Создание производных признаков

```
transformed_transactions = (transformed_transactions
    .withColumn("hour_of_day", F.hour("timestamp"))
    .withColumn("day_of_week", F.dayofweek("timestamp"))
    .withColumn("is_weekend", F.col("day_of_week").isin([1, 7]))
    )
```

Задание для студента: Реализовать преобразования согласно варианту задания (агрегация, обработка временных рядов, bucketing и т.д.).

Этап 4. Объединение данных из нескольких источников

Объединение транзакций с данными пользователей

```
enriched_transactions = (transformed_transactions
    .join(
        raw_users,
        on="user_id",
        how="left"
    )
    )
```

Обогащение данными о событиях

```
fully_enriched = (enriched_transactions
    .join(
        raw_events,
        on=["user_id", "timestamp"],
        how="left"
    )
    )
```

```
)  
)
```

```
# Анализ результатов объединения  
print(f"Enriched dataset count: {fully_enriched.count()}")  
fully_enriched.printSchema()
```

Задание для студента: Обосновать выбор типа JOIN для каждого источника данных. Какие проблемы могут возникнуть при распределённом выполнении JOIN (data skew, shuffle)?

Этап 5. Проверки качества данных (Data Quality)

```
def calculate_data_quality(df, key_columns):
```

```
    """
```

```
    Рассчитывает показатели качества данных.
```

```
    Args:
```

```
        df: DataFrame для анализа.
```

```
        key_columns: Список ключевых колонок.
```

```
    Returns:
```

```
        dict: Словарь с метриками качества.
```

```
    """
```

```
    total_rows = df.count()
```

```
    quality_metrics = {  
        "total_rows": total_rows,  
        "completeness": {},  
        "uniqueness": {},  
        "null_counts": {}  
    }
```

```
    # Проверка полноты
```

```
    for col in key_columns:
```

```
        null_count = df.filter(F.col(col).isNull()).count()
```

```
        quality_metrics["null_counts"][col] = null_count
```

```

        quality_metrics["completeness"][col] = (
            (total_rows - null_count) / total_rows * 100
        )

# Проверка уникальности ключевых колонок
for col in key_columns:
    unique_count = df.select(col).distinct().count()
    quality_metrics["uniqueness"][col] = (
        unique_count / total_rows * 100
    )

return quality_metrics

# Расчёт метрик качества
key_columns = ["transaction_id", "user_id", "timestamp", "amount"]
quality_metrics = calculate_data_quality(fully_enriched, key_columns)

for metric_name, metric_value in quality_metrics.items():
    print(f"\n{metric_name}:")
    print(metric_value)

```

Задание для студента: Реализовать дополнительные проверки качества согласно бизнес-правилам (например, проверка диапазонов значений, согласованности между колонками).

Этап 6. Загрузка в целевое хранилище (Apache Iceberg)

```

# Создание целевой таблицы Iceberg с партиционированием
target_table_name = "dds_enriched_transactions"

# Запись в Iceberg с партиционированием
(fully_enriched
 .writeTo(target_table_name)
 .using("iceberg")
 .tableProperty("format-version", "2")
 .tableProperty("write.parquet.compression-codec", "zstd"))

```

```

        .partitionedBy(F.years("timestamp"), F.months("timestamp"))
        .createOrReplace()
    )

```

Проверка загрузки

```

dds_table = spark.table(target_table_name)
print(f"DDS table count: {dds_table.count()}")
dds_table.printSchema()

```

Анализ структуры партиций

```

spark.sql(f"""
    SELECT
        years(timestamp) as year,
        months(timestamp) as month,
        count(*) as records_count
    FROM {target_table_name}
    GROUP BY years(timestamp), months(timestamp)
    ORDER BY year, month
""").show()

```

Задание для студента: Экспериментировать с различными схемами партиционирования и оценить их влияние на производительность запросов.

Этап 7. Анализ эффективности конвейера

Чтение статистики выполнения последней операции

Анализ через Spark UI или программно

Проверка идемпотентности — повторная запись

```

(fully_enriched
    .writeTo(target_table_name)
    .using("iceberg")
    .overwriteFilter(F.lit(False))
    .append()
)

```



```
print(f"DDS table count after re-run: {spark.table(target_table_name).count()}")
```

```
# Time Travel — доступ к предыдущей версии
```

```
spark.sql(f"""  
    SELECT count(*) as current_count  
    FROM {target_table_name}  
""").show()
```

4. Требования к отчёту

Отчёт должен содержать:

1. Титульный лист (ФИО студента, группа, дата выполнения)
2. Цель и задачи работы
3. Описание исходных данных (схема, объём, характеристики каждого источника)
4. Описание реализованного ETL-конвейера:
 - Этапы очистки данных с обоснованием выбора стратегий
 - Преобразования и оконные функции
 - Схема объединения данных (ER-диаграмма)
 - Проверки качества данных
5. Результаты обработки:
 - Примеры данных до и после обработки
 - Метрики качества данных
 - Объём обработанных данных
 - Структура партиционирования целевой таблицы
6. Код выполнения работы (Jupyter Notebook)
7. Выводы по результатам работы
8. Ответы на контрольные вопросы

Формат сдачи:

- Jupyter Notebook (.ipynb) с выполненным кодом
- Отчёт в формате PDF или Markdown

- Загрузка в Git-репозиторий

5. Критерии оценивания

Критерий	Баллы	Описание
Настройка окружения	10	Корректная настройка Spark-сессии с Iceberg
Загрузка данных	10	Корректное чтение из нескольких источников
Очистка данных	15	Обработка пропусков, приведение типов, удаление дубликатов
Преобразование данных	15	Оконные функции, производные признаки
Объединение данных	15	Корректное использование JOIN, обоснование выбора
Проверки качества данных	10	Расчёт метрик DQ, интерпретация результатов
Загрузка в DDS	10	Корректная запись в Iceberg с партиционированием
Отчёт	5	Полнота, структура, выводы
Защита (ответы на вопросы)	10	Понимание теоретических основ
Итого	100	

Шкала оценивания:

- 90-100 баллов: «Отлично»
- 76-89 баллов: «Хорошо»
- 61-75 баллов: «Удовлетворительно»
- Менее 61 балла: «Неудовлетворительно»

6. Правила использования внешних ресурсов и генеративного ИИ

Разрешено:

- Использование официальной документации Apache Spark, Iceberg, Hadoop

- Консультации с генеративным ИИ (ChatGPT, YandexGPT, GigaChat)

для:

- Объяснения синтаксиса PySpark
- Поиска оптимальных стратегий обработки данных
- Генерации примеров кода для ETL-процессов
- Проверки логики реализации

Запрещено:

- Полная генерация кода ИИ без понимания и модификации
- Копирование решений других студентов

- Использование готовых решений из открытых репозиториях без адаптации

Требования:

- Все запросы к ИИ должны быть задокументированы в отчёте
- Студент должен объяснить логику каждого этапа работы
- При защите необходимо продемонстрировать понимание реализованного кода

7. Список литературы

1. Захария М., Венделл П., Конвински Э., Карау Х. Изучаем Spark. Молниеносный анализ данных. — М.: ДМК Пресс, 2015. — 400 с.
2. Ральф Кимбалл, Марджи Росс. Инструментарий хранения и анализа данных. Полное руководство по размерному моделированию. — М.: Бомбора, 2021. — 656 с.
3. B. Harenslak, J. de Ruiter. Data Pipelines with Apache Airflow. — New York: Manning Publications, 2021. — 480 p.
4. Клеппманн М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. — Астана: Sprint Book, 2024. — 640 с.
5. Официальная документация Apache Spark [Электронный ресурс]. — URL: <https://spark.apache.org/docs/latest/>
6. Официальная документация Apache Iceberg [Электронный ресурс]. — URL: <https://iceberg.apache.org/docs/latest/>

8. Необходимые гиперссылки для лабораторной работы.

1. Датасет:

- US Used Cars Dataset:
https://www.kaggle.com/datasets/ananyamital/us-used-cars-dataset?select=used_cars_data.csv

2. Документация по технологиям:

- Apache Spark (официальная документация):
<https://spark.apache.org/docs/latest/>

- PySpark API (Python):
<https://spark.apache.org/docs/latest/api/python/>

- Apache Iceberg (официальная документация):
<https://iceberg.apache.org/docs/latest/>

- Apache Hadoop (официальная документация):
<https://hadoop.apache.org/docs/>

3. Для локального разворачивания окружения

Docker-образы:

- Apache Spark: <https://hub.docker.com/search?q=spark&type=image>
- Hadoop: <https://hub.docker.com/search?q=hadoop&type=image>
- Apache Iceberg: <https://hub.docker.com/search?q=iceberg&type=image>

Готовые решения для локального запуска:

- Spark + Iceberg в Docker:
<https://github.com/apache/iceberg/tree/master/examples>
- Локальный кластер Hadoop: <https://github.com/big-data-europe/docker-hadoop>

4. Инструменты для анализа данных:

Great Expectations (проверка качества данных):
<https://greatexpectations.io/>

Apache Airflow (оркестрация): <https://airflow.apache.org/>

Trino (аналитические запросы): <https://trino.io/>

Примечания для преподавателя

Методические рекомендации по организации занятия

1. Подготовительный этап (до занятия):

- Проверить доступность кластера Hadoop/Spark

- Подготовить наборы сырых данных для загрузки (минимум 3 источника)

- Развернуть тестовую базу данных в Iceberg

2. Вводная часть (0.5 часа):

- Краткий обзор архитектуры Data Lake и DDS
- Демонстрация примера ETL-конвейера
- Разбор типичных ошибок

3. Основная часть (3 часа):

- Самостоятельная работа студентов
- Консультации по индивидуальным вопросам
- Промежуточная проверка результатов

4. Заключительная часть (0.5 часа):

- Демонстрация результатов
- Обсуждение проблемных моментов
- Анонс следующего занятия

Типичные ошибки студентов

1. Неправильная настройка SparkConf — отсутствие пакетов Iceberg, некорректные пути в HDFS
2. Игнорирование приведения типов — работа со строками вместо числовых/временных типов
3. Неправильный выбор типа JOIN — использование INNER JOIN вместо LEFT JOIN приводит к потере данных
4. Отсутствие обработки NULL-значений — падение агрегаций и оконных функций
5. Игнорирование data skew — неоптимальное партиционирование приводит к "горячим" партициям
6. Неправильная схема партиционирования — слишком мелкое или крупное партиционирование

Дополнительные задания для продвинутых студентов

1. Реализовать инкрементальную загрузку данных
2. Добавить мониторинг качества данных с использованием Great Expectations
3. Реализовать кастомные UDF (User Defined Functions) для преобразования
4. Оптимизировать производительность через broadcast join и кэширование
5. Интегрировать конвейер с Apache Airflow для оркестрации.
6. Написать фрагмент кода на PySpark Structured Streaming, который имитирует чтение новых данных из директории HDFS в режиме потока (spark.readStream) и выполняет их инкрементальную запись (append) в созданную Iceberg-таблицу (writeStream), настроив директорию для checkpointing.
7. Выполнить запросы к историческим версиям Iceberg-таблицы (Time Travel) с использованием синтаксиса VERSION AS OF и TIMESTAMP AS OF. Продемонстрировать сценарий отката данных (rollback).
8. Реализовать схемную эволюцию (Schema Evolution) в Iceberg-таблице: добавление, удаление и переименование колонок через ALTER TABLE. Проверить доступность исторических данных после изменения схемы.
9. Подключиться к Iceberg-таблице через Apache Trino и выполнить аналитический SQL-запрос. Сравнить производительность с Spark SQL.
10. Проанализировать выполнение ETL-конвейера через Spark UI, выявить «узкие места» (shuffle, data skew) и предложить минимум 2 оптимизации.